FIT3182 A3 - Interactive Visualisation

by Jason Lee (34900403) and Wee Jun Zen (33524815)

Selected A3 theme and how it extends A2

Theme: Interactive visualisation

A2 Visualizations	**A3 Interactive Visualisations**
Jupyter notebook output	Interactive web dashboard with dropdown options of different data plots (Martín García-Marcos, 2025)
Static plot	Dynamic, updating plot with zoom in and out functions (Martín García-Marcos, 2025)
Line Plots ● number of violations per time ● average speed per time with max and min annotations	Bar chart Plots (Top 10): ● All live violations ● High speed offenders (>140km/h) ● Extreme speed/ reckless (> 160km/h) ● Chronic serial offenders (+10 infractions)

Relevant Research/ Technical literature

Programming an Online Dashboard to Generate Scenarios in the OpenMASTER Energy Model (Martín García-Marcos, 2025)	A Modular Python-Based Framework for Real-Time Enterprise KPI Visualization Using Pandas and Interactive Dashboards (Yaganti, 2020)
Interactive Dashboard with Dash and Plotly	Panda for data processing
Reactive Callback system	Matplotlib for static charts
Slider dropdown selection	Plotly for responsive, browser-based dashboards and real time updates
Scenario Selection via Sliders	Dash handles user interaction and config files.
	Architecture follows data ingestion, data processing, visualization and user interaction

Innovation/ technical complexity

Summary:
A live update interactive dashboard visualizing informative data without manual query from mongodb

Innovation

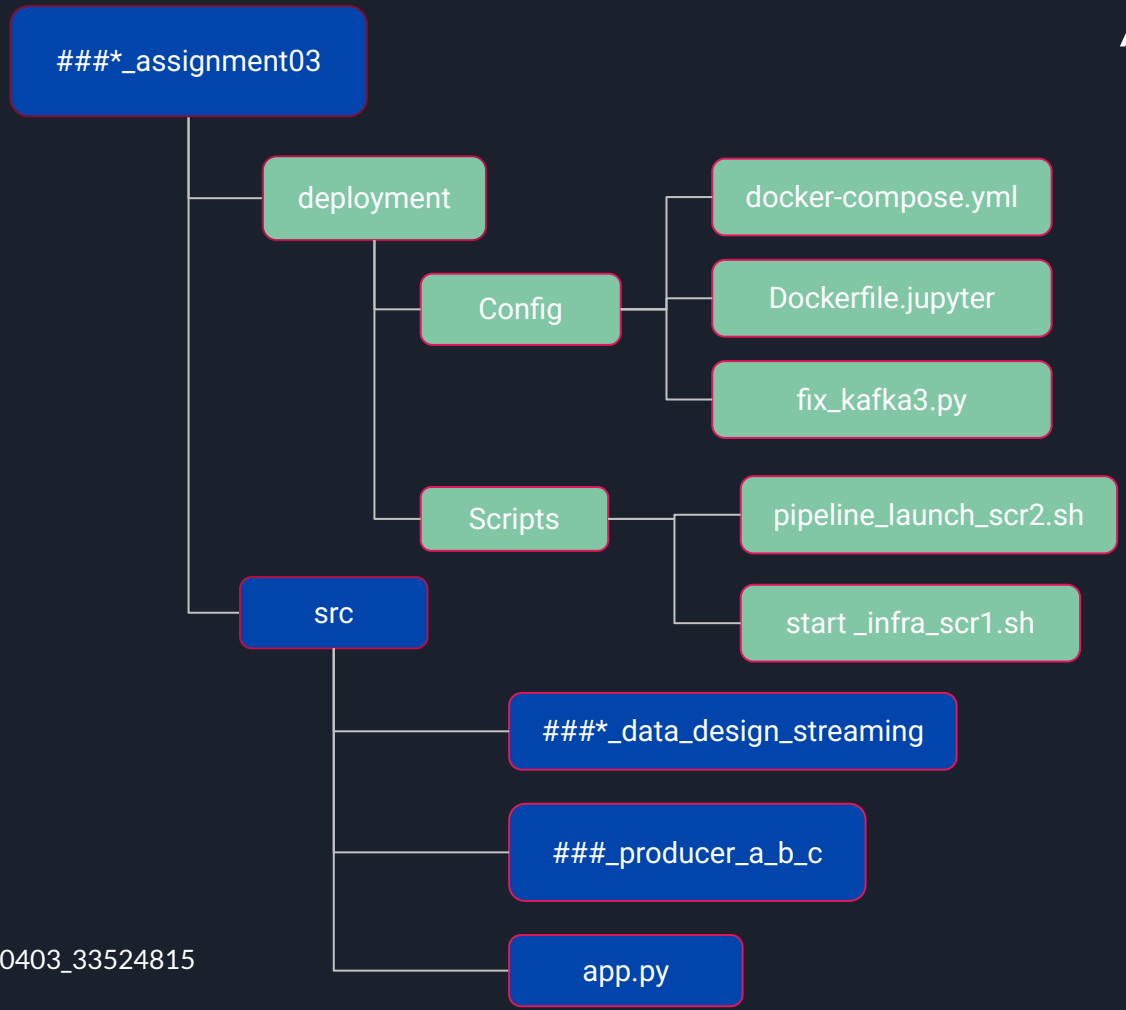
- Interactive Dashboard
- Live updates
- Data to charts end-to-end pipeline
- Dynamic color encoding

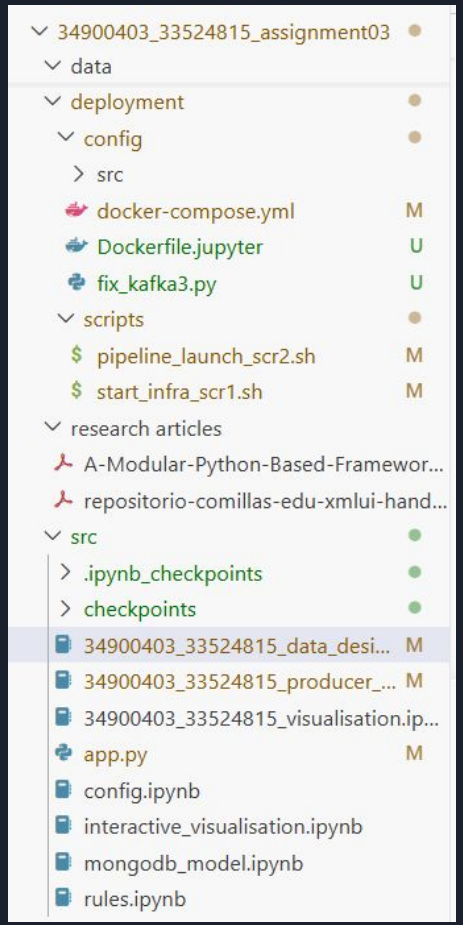
Technical Complexity

- Reactive call back system
- MongoDB live querying
- Speed limit threshold line
- Dropdown Interactivity
- Bar chart

Project demonstration

Architecture





*### refers to 34900403_33524815

Design

Data pipeline (Yaganti, 2020) :

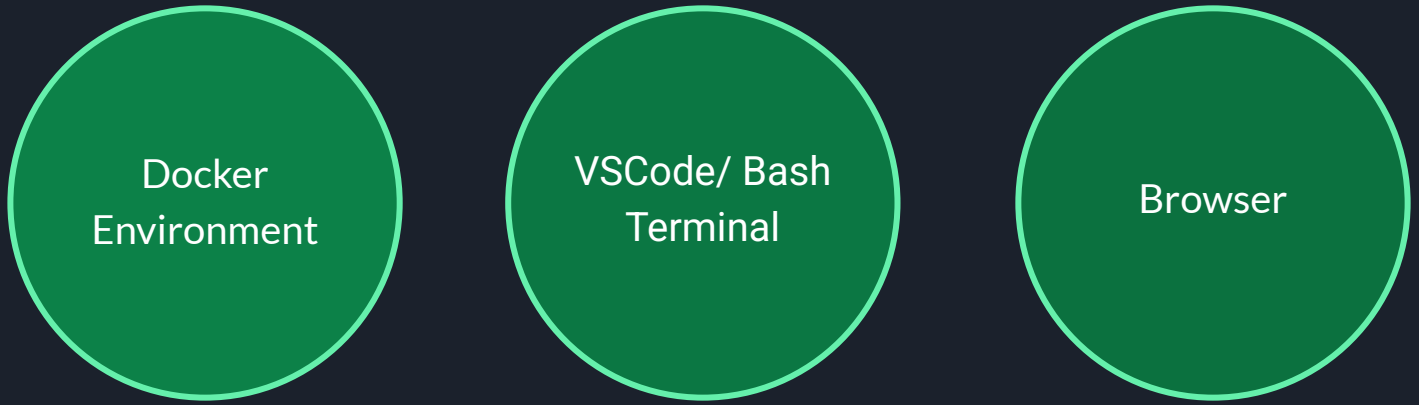
- Read from CSV
- Produce data as topics to Spark

- Process data streams and join them
- Write results to MongoDB

- Stores data
- Supplies data to Plotly and dash for visualisations

- Renders interactive web dashboard
- Provides dynamic, updating charts

Deployment



Reproducibility

Downloading prerequisites

- Docker Desktop installed and running (Windows or MacOS)
- Apache Spark version v3.5.0 or higher
- Python 3.10 or higher with access to the pip package manager

Necessary python packages:

- numpy
- pymongo
- plotly
- dash
- pandas
- jupyter
- papermill

Running app

In terminal, run in order:

1. infra_scr1.sh to start necessary docker container
2. Pipeline_launch_scr2.s to trigger the data processing pipeline

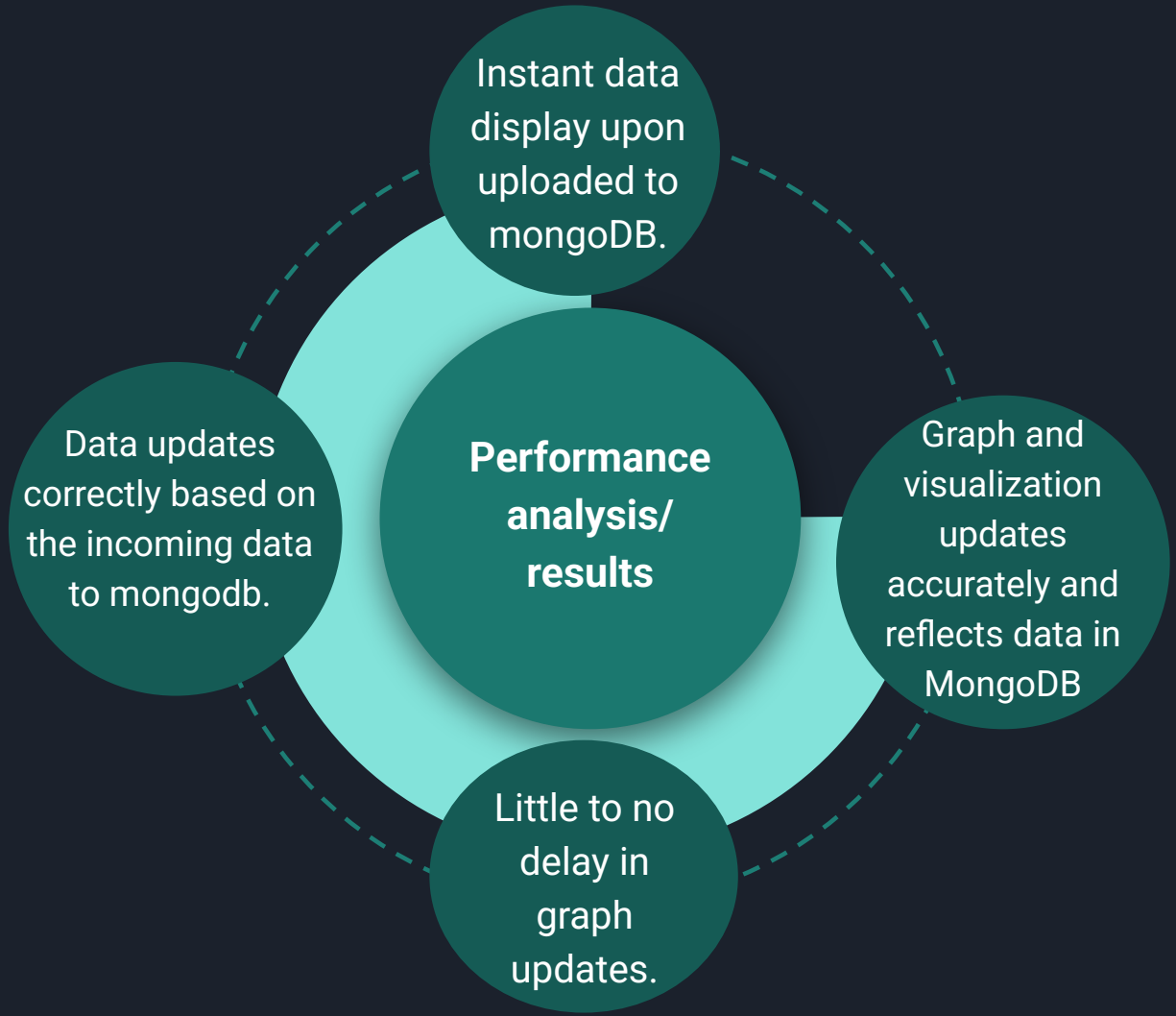
Running app

- Run python app.py in src folder

Accessing dashboard

The web app can be accessed via:

- terminal where site link is printed out
- site link "http://127.0.0.1:8050/ " in browser


Instant data display upon uploaded to mongoDB.
Graph and visualization updates accurately and reflects data in MongoDB
Little to no delay in graph updates.
Data updates correctly based on the incoming data to mongodb.
Performance analysis/ results

Justification and trade-off

Justification:

- Interactive dashboard:
  - engaging with the data
  - Easier interpretation
- Real time updates:
  - easily access most recent data.
  - See how data changes
  - identify patterns with time.

Trade off:

- Live server may go down
- Server lag due to high data throughput
- Missed updates due to aggressive user actions.
- Reliant on spark streaming processing speed

Limitations and future improvements

Limitations:

- Limited data visualization
- Limited data chart topic
- Server hosting and running
- Server lag to high data amount
- Descriptive visual analytics only

Future Improvements:

- Wider variety of visualizations
- Wider variety of data chart topics:
- Localized application
- Large data management
- Predictive modelling and AI insights

Thank you for watching!

AI Declaration Statement

We used the following AI tools:

1. Gemini
2. Claude

To help with brainstorming ideas for our interactive dashboard, learning to use tools like plotly and dash for our implementation and debugging code.

Claude and Gemini AI were also used to assist in brainstorming implementation ideas and assist in developing slide points, alongside summarizing lengthy research thesis.

The prompts used were to figure out how to implement our extension, brainstorm ideas on extension features and points for the presentation slides, debug our code and summarize the thesis and provide its key points.

We reviewed, tested and verified the final submitted work.

References

1. Martín García-Marcos, V. (2025). *Programming an Online Dashboard to Generate Scenarios in the OpenMASTER Energy Model* [Final year undergraduate thesis, Universidad Pontificia Comillas]. Repositorio de la Universidad Pontificia Comillas. http://hdl.handle.net/11531/95212
2. Yaganti, D. (2020). A Modular Python-Based Framework for Real-Time Enterprise KPI Visualization Using Pandas and Interactive Dashboards. *International Journal of Science and Research (IJSR)* , *9*(3), 1735–1738. https://dx.doi.org/10.21275/SR20033094751